

Practical Exercises and Homework

To reinforce the concepts from Session 7, we'll end with take-home **homework activities**. These include a mix of *tick-box questions* (to self-check understanding) and *hands-on exercises* (to apply the knowledge). Participants are encouraged to try these before the next session:

1. Knowledge Check – Tick the Boxes

Below is a list of statements related to preparing Power Platform and Azure environments. Mark each as **True** or **False** (✓ or ✗) based on what you learned:

- ☐ **Statement:** *Every user in the tenant should have permission to create production Power Platform environments.*
Answer: False – Environment creation should be limited to admins or an approval process to prevent sprawl. [\[Power Plat...d strategy\]](#)
- ☐ **Statement:** *Data Loss Prevention policies in Power Platform can restrict which connectors are allowed to be used together in an app or flow.*
Answer: True – DLP policies classify connectors and prevent mixing Business vs Non-Business data connectors, helping protect sensitive info. [\[Enterprise...r.Platform\]](#)
- ☐ **Statement:** *An Azure Landing Zone for AI should address network security (e.g., setting up VNets, firewalls) from the start.*
Answer: True – A key principle of landing zones is to configure networking (with things like DDoS Protection, private endpoints, etc.) upfront for secure connectivity. [\[AI Ready - ...soft Learn\]](#)
- ☐ **Statement:** *Custom Copilot extensions allow Copilot to retrieve information from or take actions in external systems.*
Answer: True – Copilot extensions (via plugins, connectors, etc.) extend Copilot's reach beyond Microsoft 365 data. [\[Build your...extensions\]](#)
- ☐ **Statement:** *Power Platform's CoE Starter Kit is primarily a set of training documents for end users.*
Answer: False – The CoE Starter Kit is a collection of apps/flows for admins to govern and monitor the Power Platform adoption (e.g., tracking apps, automating governance tasks). [\[Microsoft...E Tool Kit\]](#)

Review your answers with the supplied answer key (we will provide this on the session page). If you find any incorrect, revisit the relevant section of the report above to refresh the concept.

2. Hands-On Exercise – Power Platform Environment Setup

Goal: Set up a **Trial Power Platform environment** and configure a basic Data Loss Prevention policy.

Instructions:

- a. **Create a Trial Environment:** If you have Power Platform admin privileges (or can get a temporary trial license), navigate to the Power Platform Admin Center. Create a new **Trial** environment (these are free for 30 days and great for learning). Name it “Training Env”. Enable the **Dataverse** database when prompted (this might take a few minutes to provision). *If you cannot create an environment, use the existing Default environment for the next steps (with caution), or sign up for the **Power Apps Developer Plan** which gives you your personal dev environment.*
- b. **Explore Environment Settings:** In the new environment’s settings, find where you can assign **Security Roles**. Identify how to make a user an Environment Maker vs Environment Admin. (If you’re the creator, you are automatically admin.) This is just to familiarize yourself with the controls.
- c. **Create a DLP Policy:** In the Power Platform admin center, under Data Policies, create a new policy called “Training DLP”. Set the scope to just your “Training Env” (so you don’t affect others). In the policy, add a few connectors:
 - Mark one or two popular business connectors (e.g., SharePoint, SQL) as *Business* data.
 - Mark a couple of social or consumer connectors (e.g., Twitter, RSS) as *Non-Business*.
 - Leave the rest as *Default*. Save the policy.
- d. **Test the Policy:** Open Power Automate (make sure you’re creating in the “Training Env”). Try to create a simple flow that uses one connector from Business (e.g., SharePoint) and one from Non-Business (e.g., Twitter) in the same flow. The system should either warn or block you from adding both due to the DLP conflict. Observe what the error message says – this confirms your policy is active.
- e. **Clean Up:** You can delete the flow test and even the environment after testing (since it’s a trial, it will auto-expire eventually). Delete the DLP policy if you no longer need it.

What you learn: This exercise helps you get hands-on with the tools that an admin would use to implement governance on Power Platform. You’ll see how easy it is to spin up an environment (and thus why governance is needed to control that in a real org!), and you’ll experience how DLP policies enforce rules for makers.

3. Hands-On Exercise – Azure Landing Zone Planning

This is more of a planning/design exercise than building (unless you have an Azure subscription to experiment with):

- **Design a Minimal Landing Zone:** Take a hypothetical scenario – your company wants to start using Azure for AI projects. Sketch a basic architecture of an Azure AI Landing Zone on paper or a whiteboard. Identify:
 - How many subscriptions you would use and what each is for (e.g., one for shared services, one for AI dev, one for AI prod).
 - Key resource types in each (e.g., Subscription “AI-Shared” will contain a VNet, a firewall, etc.).
 - At least 3 Azure Policies you think would be important (e.g., “All storage accounts must have encryption and private endpoints” or “Disallow public IPs on VMs”).
 - How you would organize identities (Will you require service principals for apps? Will you use managed identities? How will users be granted access?).
- **Optionally, Implement a Piece:** If you have access to Azure, try implementing just one part of your design: for example, create a Management Group and two subscriptions (you can use Azure Free Trial or an MSDN subscription if available). Apply a sample Azure Policy to one of the management groups (Azure has a policy definition for requiring resource tags – you can use that for test). This will let you see the mechanics of how management groups and policies work.
- **Document Assumptions:** Write a short paragraph on why you chose that structure. For instance, “I chose to separate dev and prod to ensure experiments don’t affect production and to allow different access controls. I put a firewall in the shared resources so both dev and prod apps route through a secure point.” This will help clarify your reasoning.
- **Share & Discuss:** If you have colleagues or others also doing this exercise, compare designs. There’s no single correct answer – it depends on company size and needs. Discuss why maybe one would have more strict network isolation while another would focus on simpler setup initially.

This exercise is about *applying the abstract ideas* to a concrete scenario. Even without actual Azure deployment, the act of designing consolidates your understanding of landing zone components. It prepares you to have an informed conversation with cloud architects or to evaluate your current cloud setup.

4. Explore Copilot Extension (Research Activity)

Since custom Copilot extensions are relatively new, here's an exploratory task:

- **Visit Microsoft Copilot Studio** (if you have access to the preview, otherwise watch the Microsoft Mechanics video on “Build your own Copilot”). Note the capabilities shown – what types of extensions can you create? [\[Build your...extensions\]](#)
- **Identify a Use Case:** Think of one manual task in your daily work that could be eased by asking an AI assistant. Write it down. Now outline (in a few bullets) how a Copilot extension could be created to handle that. For example: *Use case:* “Checking inventory status in ERP when drafting an email to a client.” *Potential extension:* A plugin that queries the ERP’s API for product stock levels, triggered when Copilot sees an email mention “stock” or when user asks specifically.
- **List Required Components:** For your use case, list what you would need – e.g., need an API from ERP, need a custom connector, need user permission context, etc. This reinforces thinking through the integration points.
- **Security Thought Exercise:** Also consider what could go wrong or what you must guard against. Perhaps the ERP data is sensitive (so we need to ensure Copilot only shows it to certain roles), or there's a risk of erroneous orders if Copilot misinterprets something (so maybe the extension should never execute an action without confirmation).

This is a more open-ended, creative exercise. It's meant mostly for you to connect the dots between a real business problem and the solution architecture. It can spark ideas on how you might leverage Copilot in your own projects, and what prep work (the kind we discussed in this session) would be necessary to make it happen.

The above exercises cover theoretical and practical aspects. We encourage everyone (developers *and* business stakeholders alike) to at least review the tick-box questions and try the **Power Platform environment exercise**. Business decision makers might focus more on the planning and conceptual parts (exercises 3 and 4), while developers/IT pros will enjoy the hands-on setup in exercise 2.